Generative AI Analysis Report

Conditional Transformer-Based Generation of 2D Platformer Level Segments

Student: Robert Mayfield **Project:** Udacity AI Masters Capstone — Generative AI Applications

---

Overview

This project implements and evaluates a conditional generative model for 2D platformer level segment synthesis. The work sits within the broader field of procedural content generation, the algorithmic creation of game content such as levels, maps, and items (Shaker et al., 2016). The model is a decoder-only Transformer trained on tile-based level data from the Video Game Level Corpus (VGLC), with the objective of generating novel level segments that align with a requested difficulty target: Easy, Medium, or Hard. Difficulty conditioning is implemented through prepended control tokens that prompt the model to generate structurally appropriate tile sequences.

The primary research question is whether a small autoregressive Transformer can learn the structural associations between difficulty labels and tile configurations well enough to produce directed generation, given the small corpus size available in the VGLC. Secondary questions concern the effect of corpus size on generation quality and the effectiveness of alternative sampling strategies for improving output novelty.

The project trains a conditional model and a baseline unconditional model, evaluates them on four metrics (structural validity, diversity, nearest-neighbour similarity, and difficulty match accuracy), and extends the analysis with a corpus expansion experiment and a sampling strategy experiment. Key results show strong difficulty alignment (95.3% baseline accuracy, 98.7% with the expanded corpus) but near-verbatim reproduction of training data in most outputs, with mean nearest-neighbour tile similarity above 0.99.

Dataset Description

The primary data source is the Video Game Level Corpus (VGLC), an open academic dataset of tile-based platformer levels (Summerville et al., 2016). The VGLC encodes each level as a grid of tile characters representing gameplay elements: ground, enemies, pipes, blocks, gaps, and collectibles. This project uses Super Mario Bros (SMB1) overworld levels as the baseline training corpus, with Super Mario Bros: The Lost Levels (SMB2J) added in the corpus expansion experiment.

Baseline corpus: 15 SMB1 overworld levels downloaded from the VGLC repository. Levels range from 149 to 373 columns wide and are uniformly 14 rows tall. Total tile count across the corpus is 40,922 tiles.

Expanded corpus: 22 Lost Levels files were downloaded from the VGLC. Of these, 20 were retained after normalization: files with non-standard row heights had leading all-sky rows trimmed to reach the 14-row standard, and 2 files (12 and 13 rows) that could not be normalized by sky trimming alone were excluded. The expanded corpus contains 35 levels total.

The tile vocabulary contains 13 characters: the empty sky tile (`-`), ground (`X`), breakable blocks (`S`), enemies (`E`), pipes (`<`, `>`, `[`, `]`), question blocks (`?`, `Q`), cannon tiles (`B`, `b`), and coins (`o`). The sky

tile dominates at 86.21% of all tile positions. Ground tiles account for 8.80%, and all interactive and hazard tiles together account for less than 3%.

Each level is split into overlapping 14-row by 32-column chunks with a stride of 16 columns. This produces 161 chunks from the baseline corpus and 390 chunks from the expanded corpus. Each chunk contains 448 tile positions and represents approximately two screens of horizontal gameplay.

The dataset is committed directly to the repository under `data/raw/` and requires no external download to reproduce the analysis.

Model Design and Training Approach

Architecture. Both the conditional and baseline models use a decoder-only Transformer architecture for next-token prediction (Vaswani et al., 2017). The model consists of 4 Transformer encoder layers with 8 attention heads, a model dimension of 256, and a feedforward sublayer of 1024 dimensions. The output projection maps from 256 dimensions to the 16-token vocabulary (13 tile characters plus 3 difficulty control tokens). Total parameter count is approximately 3.28 million for both models.

This architecture follows the approach of Sudhakaran et al. (2023), who demonstrated that autoregressive Transformers can generate playable Mario-style level segments by treating level content as a flat token sequence. The decoder-only formulation is appropriate because level generation is an autoregressive task: each tile is predicted from all preceding tiles in a left-to-right, top-to-bottom reading order.

Tokenization. Tile characters are assigned integer IDs 0 through 12 in sorted order. Three difficulty control tokens are assigned IDs 13 through 15 (Easy, Medium, Hard). Each tokenized training sequence has length 449: one control token followed by 448 tile tokens read row by row across the 14-row by 32-column chunk.

Conditioning mechanism. The conditional model receives a difficulty control token as its first input token for every training sequence. The token is prepended before the tile content and is therefore attended to by every subsequent tile prediction through the self-attention mechanism. At generation time, the model is seeded with a control token and generates tile tokens autoregressively until the full 448-tile sequence is complete. Control tokens are masked from the tile prediction distribution at every step to prevent them from appearing as tile outputs.

The baseline model is trained on the same tile sequences with the control token stripped, providing a reference point for quantifying the effect of conditioning.

Training. Both models are trained for 150 epochs using the AdamW optimizer with a learning rate of 3e-4, weight decay of 1e-2, and gradient clipping at a maximum norm of 1.0. The loss function is cross-entropy over the 16-token vocabulary. The train/validation split uses an 80/20 stratified split preserving difficulty class balance. For the baseline corpus this produces 128 training and 33 validation sequences; for the expanded corpus, 312 training and 78 validation sequences. All random seeds are fixed at SEED=42 using Python, NumPy, PyTorch CPU, PyTorch CUDA, and cuDNN determinism flags.

Training was performed on an RTX 3080 (10GB VRAM). Both models completed 150 epochs in under 5 minutes due to the small corpus size.

Preprocessing Decisions

Difficulty scoring. Difficulty labels are derived from five structural features computed per chunk: enemy density (fraction of tiles that are enemies), hazard density (fraction of tiles that are enemies or cannon tops), longest ground gap (maximum consecutive empty-tile run in the bottom row), solid tile ratio (fraction of tiles that are solid), and collectible density (fraction of tiles that are coins). A weighted difficulty score is computed as:

```
score = enemy_density × 40 + longest_gap × 5 + hazard_density × 30
```

The longest ground gap carries the dominant weight because pit navigation is the primary mechanical challenge in Super Mario Bros gameplay. The feature weights are heuristic and exploratory rather than empirically validated. This reflects a broader reality in the field: there is no consensus methodology for evaluating procedural level generation systems, and difficulty in particular lacks a standard quantitative definition (Withington et al., 2024). The scoring formula here is therefore presented as one reasonable heuristic among many rather than a ground-truth measure of difficulty.

Difficulty labels are assigned using percentile-based thresholds: chunks scoring below the 33rd percentile are labeled Easy, chunks between the 33rd and 67th percentile are labeled Medium, and chunks at or above the 67th percentile are labeled Hard. For the baseline corpus the thresholds are Easy < 5.68 and Hard >= 20.78, producing 54 Easy, 52 Medium, and 55 Hard chunks.

Percentile-based thresholds were chosen over fixed thresholds to ensure balanced class sizes regardless of corpus composition. With only 161 baseline chunks and 3 classes, severe class imbalance would prevent the conditional model from learning meaningful difficulty associations.

Corpus normalization. Lost Levels files use variable row heights (12 to 16 rows). All-sky leading rows were trimmed from 15-row and 16-row files to reach the 14-row standard. This normalization is semantically clean because all removed rows consisted entirely of the sky tile with no content. Files with 12 or 13 rows were excluded because they required padding rather than trimming, which would introduce synthetic tile content.

Output Evaluation and Interpretation

Structural validity was 100% across all experiments. Every generated segment used only the 13 valid tile characters, produced the correct 14-row by 32-column grid, and contained at least one non-empty tile. This result is expected: the generation loop masks control token IDs from the tile prediction distribution at every step, making out-of-vocabulary generation impossible by construction.

Difficulty match accuracy was 95.3% overall for the baseline model, with per-class F1 scores of 0.960 (Easy), 0.940 (Medium), and 0.960 (Hard), and a macro F1 of 0.953. These results are well above the 33% chance baseline and confirm that the difficulty control token has a measurable effect on generated output structure. Medium achieved the lowest F1 because it occupies the interval

between both score thresholds, and outputs near a threshold boundary can tip into either adjacent class with small changes in gap length or enemy placement.

Expanding the corpus to 35 levels improved overall accuracy to 98.7% and macro F1 to 0.987, with per-class F1 rising for every class (Easy 0.980, Medium 0.980, Hard 1.000). The improvement reflects the additional structural variety within each difficulty class provided by the Lost Levels data, which gives the model more diverse examples of what each control token should produce.

Diversity was 76.7% for the baseline model: 115 of 150 generated segments were distinct tile grids. Expanding the corpus improved diversity to 87.3%, confirming that more training data reduces the model's tendency to collapse onto a small set of high-probability patterns.

Nearest-neighbour similarity revealed the extent of memorization. The mean maximum tile similarity between a generated segment and its closest training chunk was 0.991 for the baseline model, meaning generated outputs differed from their nearest training match in approximately 4 tile positions out of 448. Expanding the corpus raised mean similarity slightly to 0.993 because the larger corpus covers more structural space, making it more likely that any generated output finds a close training match.

Sampling experiments showed that temperature is the most effective lever for reducing memorization behavior. At temperature 1.2, diversity reached 92.7% and mean max similarity dropped to 0.980 while difficulty match accuracy remained at 94.7%, essentially matching the baseline. At temperature 1.5, diversity reached 98.0% and mean max similarity dropped to 0.946, but accuracy fell to 84.7%. The novelty rate, the fraction of outputs more than 10% different from any training chunk, rose from under 1% at temperature 0.8 to 17.3% at temperature 1.5, the only configuration that produced a meaningful share of genuinely new structure. Temperature 1.2 represents the best practical trade-off for applications requiring varied output without severe accuracy degradation.

Top-k sampling produced modest improvements over the temperature 1.0 baseline with minimal accuracy impact, though no configuration meaningfully reduced mean max similarity. Nucleus sampling (top-p) was counterproductive for this vocabulary: at low p values the sky tile dominates the probability nucleus, removing structural tiles from the sampling distribution and collapsing diversity to 10%. This is a direct consequence of the 86% sky tile frequency and is specific to heavily skewed tile distributions.

Interpretation for a Non-Technical Audience

This project taught a computer program to create new platformer level sections by learning from existing Super Mario Bros levels. The program studied 35 levels from two Mario games and learned patterns about how tiles are arranged: where the ground is, where enemies appear, where gaps occur. It can then generate new level sections by predicting, one tile at a time, what should come next based on everything it has already placed.

The program can be given a difficulty instruction before it starts generating. When told to make an Easy level, it tends to produce sections with solid, unbroken ground and few obstacles. When told Hard, it tends to produce sections with longer ground gaps that require bigger jumps. This difficulty control works about 95% of the time on the original training data and about 99% of the time after adding the Lost Levels, when measured against the same scoring system used to label the training data.

The most important limitation is that the program mostly reproduces patterns it has already seen rather than inventing truly new designs. When we compared its outputs to the training levels tile by tile, we found that nearly every output was more than 99% identical to some chunk from the training data. The program is working more like a very organized filing system than a creative designer: it retrieves appropriate memorized patterns rather than composing new ones. This happens because the training dataset is small and the program has enough capacity to remember most of it.

Increasing the randomness in how the program makes tile selections (higher temperature) does produce more genuinely different outputs, though it also makes the difficulty control less reliable. Adding more training data from the Lost Levels improved the variety of patterns available, but did not change the fundamental tendency toward reproduction.

Failure Cases and Error Analysis

Four categories of failure were identified through systematic analysis of 150 generated segments.

Difficulty misclassification. The failure probe generated 90 samples (30 per class) and surfaced only one misclassification, consistent with the high difficulty match accuracy reported in evaluation. The single case was an Easy-conditioned output that scored Medium (score 20.0): a small cluster of question and breakable blocks plus a short ground gap pushed the score just above the Easy threshold of 5.68. This illustrates threshold boundary drift, where an output whose computed score lands near a percentile threshold is assigned to an adjacent class regardless of the conditioning token. The underlying cause is structural: the model has not learned a semantic notion of difficulty, only structural associations with control tokens that are mediated entirely by the scoring formula, so any output whose score crosses a threshold is relabelled.

Near-verbatim reproduction. Two reproduction examples were found. An Easy-conditioned output matched a chunk from mario-5-1 column 160 at 0.9955 similarity, differing in only 2 of 448 tiles. A Medium-conditioned output matched a chunk from mario-6-1 column 48 at exactly 1.0 similarity, a perfect tile-for-tile copy. This is consistent with the 0.991 mean nearest-neighbour similarity measured in the evaluation section. The model has memorized specific training sequences and retrieves them under the appropriate conditioning context rather than composing new arrangements.

Pipe integrity violations. Six of 150 samples (4.0%) contained pipe top tokens (<>) without pipe body tokens ([ ]) directly below, producing structures that are tile-vocabulary-valid but semantically impossible to render correctly. These violations arise from the autoregressive sampling process: the model predicts a pipe top token at a row where the training distribution supports it, but subsequent row predictions choose other content rather than a pipe shaft. Because the structural validity check only verifies tile vocabulary and grid dimensions, these violations pass validation.

Scoring-weight artifact in Easy samples. Eighteen of 50 Easy-scored outputs (36%) contained six or more enemy tiles. The worst example had 11 enemies with a solid ground row, scoring Easy because the longest gap was zero. This is a direct consequence of the heuristic scoring design: the gap feature dominates difficulty score by a factor of roughly 6:1 over enemy density at typical values. The model has learned that Easy conditioning correlates with solid ground rows and with certain structural patterns from the training data, some of which include high enemy counts.

Ethical Considerations and Responsible Use

Data provenance and copyright. The VGLC levels are derived from commercial video games owned by Nintendo. The VGLC is published for academic research purposes under a Creative Commons license, and its use here is consistent with that context. The levels are not reproduced or redistributed in a form that would substitute for the original games. Generated outputs are new tile grids derived from learned structural patterns, not copied level content, though the memorization analysis showed that a substantial fraction of outputs are near-identical to training chunks.

Bias in training data. The corpus is drawn entirely from two Nintendo titles from the mid-1980s, both designed for the same hardware platform and following the design conventions of a single creative team. Generated outputs reflect these conventions and cannot be expected to represent the full space of platformer level design. Levels designed for accessibility, for different demographic audiences, or for contemporary design conventions are absent from the training distribution. Any system that uses this generator to make decisions about player experience should account for the narrow design vocabulary it has learned.

Heuristic difficulty labels. The difficulty labels used to train the conditional model are computed from structural features, not from player data. A level labeled Hard by this system may not feel challenging to an experienced player, and a level labeled Easy may be impassable for a player with limited motor skills or game experience. The labels encode a specific technical definition of difficulty tied to gap length and enemy density, not a general psychological or accessibility-based assessment. This distinction must be communicated clearly in any downstream application that surfaces difficulty labels to users or uses them to make adaptive decisions.

Deployment considerations. The model is intended as a research prototype for exploring conditional generation. It should not be deployed in a production system without human review of generated outputs, validation against a broader set of difficulty criteria, and disclosure to end users that content is algorithmically generated. Automated suppression or promotion of generated level content based solely on classifier output without human oversight would be inappropriate. The memorization behavior identified in evaluation means that deployed outputs may closely replicate training data, raising additional questions about appropriate use in commercial contexts.

Limitations and Future Improvements

Corpus size and memorization. The most significant limitation is the small training corpus. With 128 to 312 training sequences and a 3.28 million parameter model, the model has sufficient capacity to memorize training data rather than learn generalizable structural rules. Mean nearest-neighbour tile similarity of 0.99 at the default sampling temperature confirms this, and only aggressive high-temperature sampling reduces it appreciably. Addressing memorization would require substantially more training data, a smaller model relative to corpus size, or stronger regularization. The masked-loss approach — removing the sky tile from the training signal while retaining it as context — is a candidate near-term experiment that requires no architectural changes and could redirect model capacity toward the 14% of tiles that define level structure.

Heuristic difficulty scoring. The gap-dominated scoring formula creates a systematic mis-

match between the difficulty label and intuitive difficulty: Easy-labeled outputs can contain many enemies, and Hard-labeled outputs without gaps can be structurally benign. Player-validated difficulty labels from playtesting data would be a more reliable signal, but no such dataset exists for the VGLC corpus at the chunk level.

Vocabulary skew. The 86% sky tile frequency creates practical problems for sampling: nucleus sampling is rendered ineffective because the sky tile dominates the probability nucleus at any reasonable p value. This limits the applicability of standard language model decoding strategies without modification. A run-length encoding of sky regions or a masked-loss training approach would reduce this imbalance.

Single-game domain. All training data comes from two games sharing the same engine. The model has learned Mario-specific structural conventions (horizontal ground rows, specific pipe and pipe-body paired tiles, specific enemy placement conventions) that may not transfer to other platformer styles. Expanding to other VGLC games or to custom-designed levels would broaden the model's structural vocabulary, though it would require handling vocabulary differences across games.

Generation coherence. The autoregressive model predicts tiles one at a time without explicit awareness of long-range structural constraints. Pipe tops should always be followed by pipe bodies; ground gaps should be jumpable given the player's physics parameters; enemy placements should relate to platform positions. The model captures some of these regularities through statistical co-occurrence but can produce locally valid but globally inconsistent structures, as seen in the pipe integrity violations identified in the failure analysis.

References

Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T. (2019). Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency* (pp. 220–229). ACM. https://doi.org/10.1145/3287560.3287596

Shaker, N., Togelius, J., & Nelson, M. J. (2016). *Procedural content generation in games*. Springer. https://doi.org/10.1007/978-3-319-42716-4

Sudhakaran, S., Gonzalez-Duque, M., Freiberger, M., Glanois, C., Najarro, E., & Risi, S. (2023). MarioGPT: Open-ended text2level generation through large language models. In *Advances in Neural Information Processing Systems* (Vol. 36). https://arxiv.org/abs/2302.05981

Summerville, A., Snodgrass, S., Mateas, M., & Ontanon, S. (2016). The VGLC: The video game level corpus. In *Proceedings of the 7th Workshop on Procedural Content Generation*. https://arxiv.org/abs/1606.07487

Summerville, A., Snodgrass, S., Guzdial, M., Holmgård, C., Hoover, A. K., Isaksen, A., Nealen, A., & Togelius, J. (2018). Procedural content generation via machine learning (PCGML). *IEEE Transactions on Games, 10*(3), 257–270. https://doi.org/10.1109/TG.2018.2846639

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems* (Vol. 30). https://arxiv.org/abs/1706.03762

Withington, O., Cook, M., & Tokarchuk, L. (2024). On the evaluation of procedural level generation systems. In *Proceedings of the 19th International Conference on the Foundations of Digital Games*. https://doi.org/10.1145/3649921.3650016